מסמך עיצוב | Model Kill-Switch Protocol

מהארכיטקטורה אל הקוד: אילו מחלקות יש, איך כל פקודה עוברת ביניהן, איזה מצב היא יכולה להיות בו, מי רץ במקביל ומה נשמר.

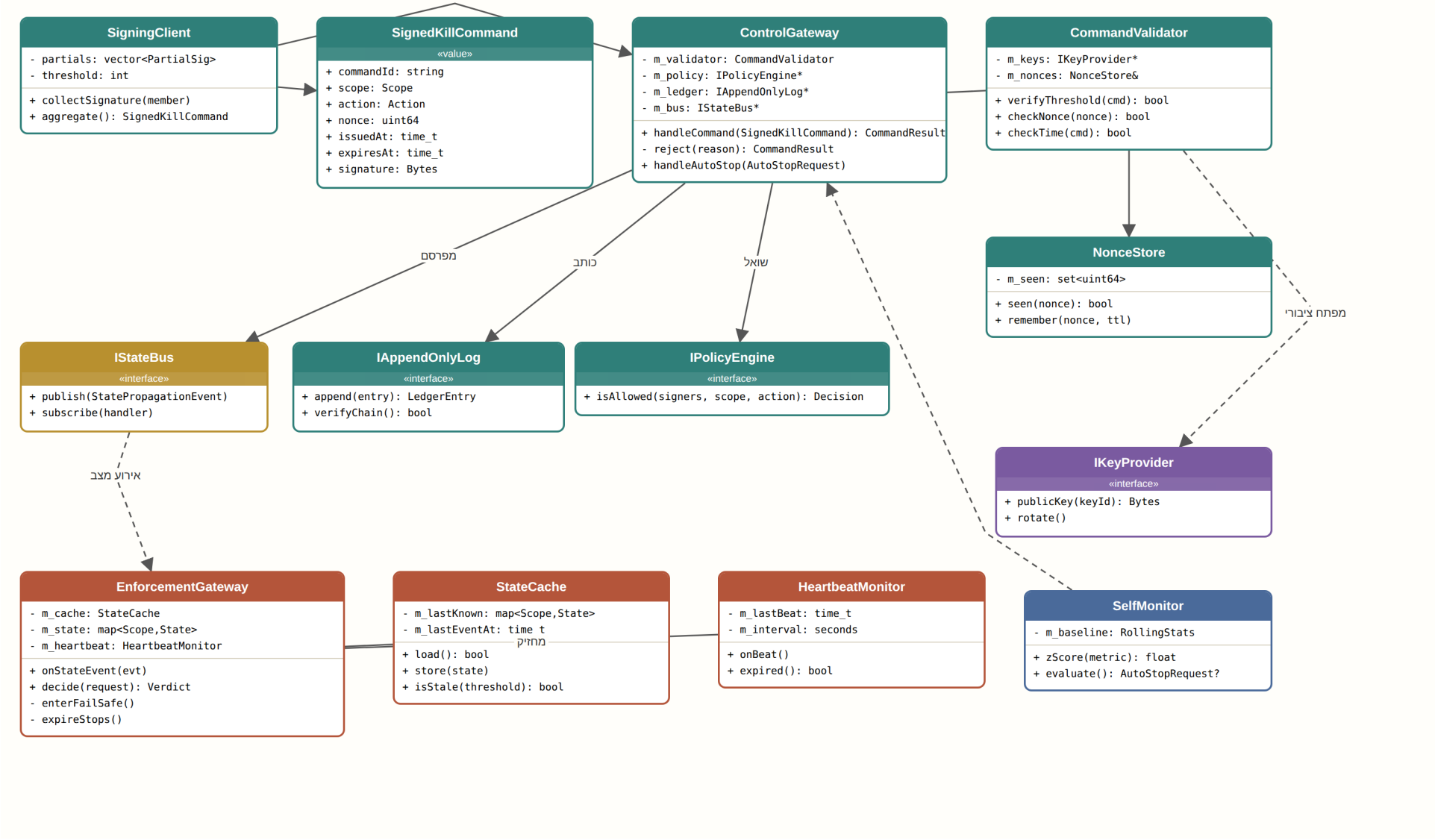
מסמך העיצוב בא אחרי הארכיטקטורה. הארכיטקטורה ענתה על השאלה "אילו רכיבים יש ומי מדבר עם מי". כאן יורדים רמה אחת: כל רכיב נהפך למחלקות עם שדות ופעולות, ומוגדר מה עובר בין המחלקות. כך אפשר לכתוב קוד בלי לנחש.

1. עקרונות העיצוב

- תלות בממשק ולא במימוש: כל מה שיכול להתחלף (מדיניות, יומן, אפיק, מפתחות) הוא ממשק, כדי שאפשר לבדוק כל רכיב לבד ולהחליף טכנולוגיה בלי לשנות את השער.
- כיוון אחד של שליטה: פקודות נכנסות רק דרך ControlGateway. אין מסלול עוקף, ולכן יש נקודה אחת לבדוק ולהגן עליה.
- מצב אחד, כמה עותקים: המצב האמיתי נקבע במרכז, ולכל שער יש עותק מקומי. כשהם לא מסכימים, השער פועל לפי כלל כשל בטוח שנקבע מראש.
- כל דחייה נשמרת: דחייה היא מידע, לא רעש. היא נרשמת ביומן האבטחה.

2. דיאגרמת המחלקות

כל קופסה היא מחלקה. החלק העליון הוא השם, אחריו השדות (מה היא מחזיקה), ואחריהם הפעולות (מה היא יודעת לעשות). קופסה עם «interface» היא חוזה בלבד: היא קובעת אילו פעולות חייבות להיות, ומי שמממש אותה מחליט איך. חץ מלא הוא "משתמש ב", חץ מקווקו הוא העברת הודעה או בקשה, ויהלום הוא "מחזיק".



דיאגרמת המחלקות

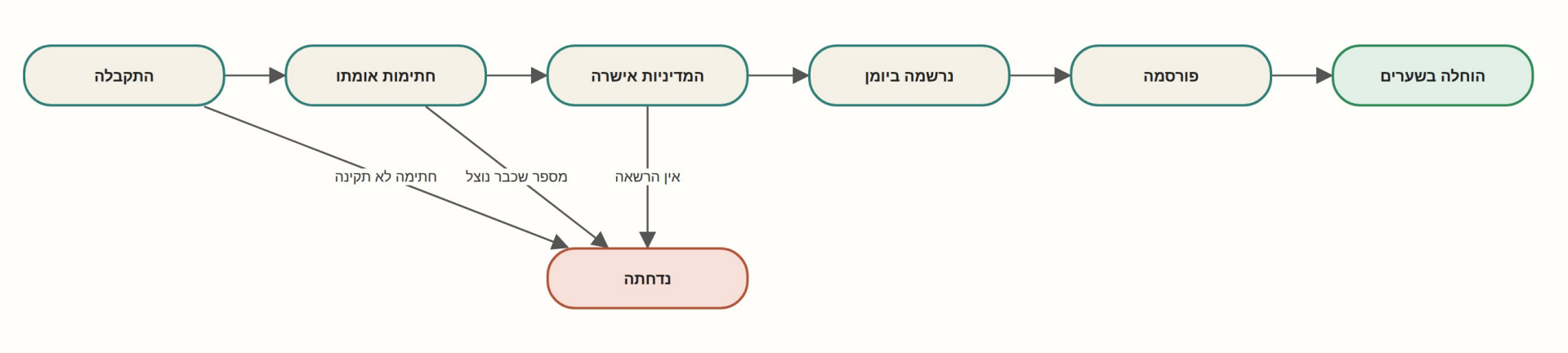
מה כל מחלקה עושה

מחלקה	תפקיד
SigningClient	אוסף חתימות חלקיות מחברי הוועדה ומאחד לפקודה אחת.
SignedKillCommand	אובייקט ערך: הפקודה עצמה, עם הנתונים שמקשרים אותה לתחום, לפעולה ולזמן. אי אפשר לשנות אותה אחרי החתימה.
ControlGateway	הדלת היחידה. מתזמר את הבדיקות ואת השלבים אחריהן, ולא מחליט בעצמו.
CommandValidator	בודק רק שלושה דברים: סף החתימות, מספר חד פעמי וזמן.
NonceStore	זוכר מספרים שכבר נוצלו, כדי שאי אפשר לשחזר פקודה ישנה.
IPolicyEngine	ממשק למנוע מדיניות. המימוש יכול להחליף כללים בלי לגעת בשער.
IAppendOnlyLog	ממשק ליומן שרק מוסיפים אליו. כל רשומה מצביעה על הקודמת בגיבוב.
IStateBus	ממשק לאפיק ההפצה. מפרסמים פעם אחת וכל השערים מקבלים.
IKeyProvider	ממשק לכספת המפתחות. אף רכיב לא מחזיק מפתח בעצמו.
EnforcementGateway	השומר ליד השרת. מקבל אירועי מצב, מחליט על כל בקשה ועובר למצב כשל בטוח כשהקשר נפגע.
StateCache	עותק מקומי של המצב האחרון הידוע, שנשמר גם כשהרשת נופלת.
HeartbeatMonitor	בודק שהפעימות מהמרכז ממשיכות להגיע.
SelfMonitor	מחשב סטייה מהרגיל ויכול רק לבקש עצירה, דרך אותו שער.

3. מכונות מצבים

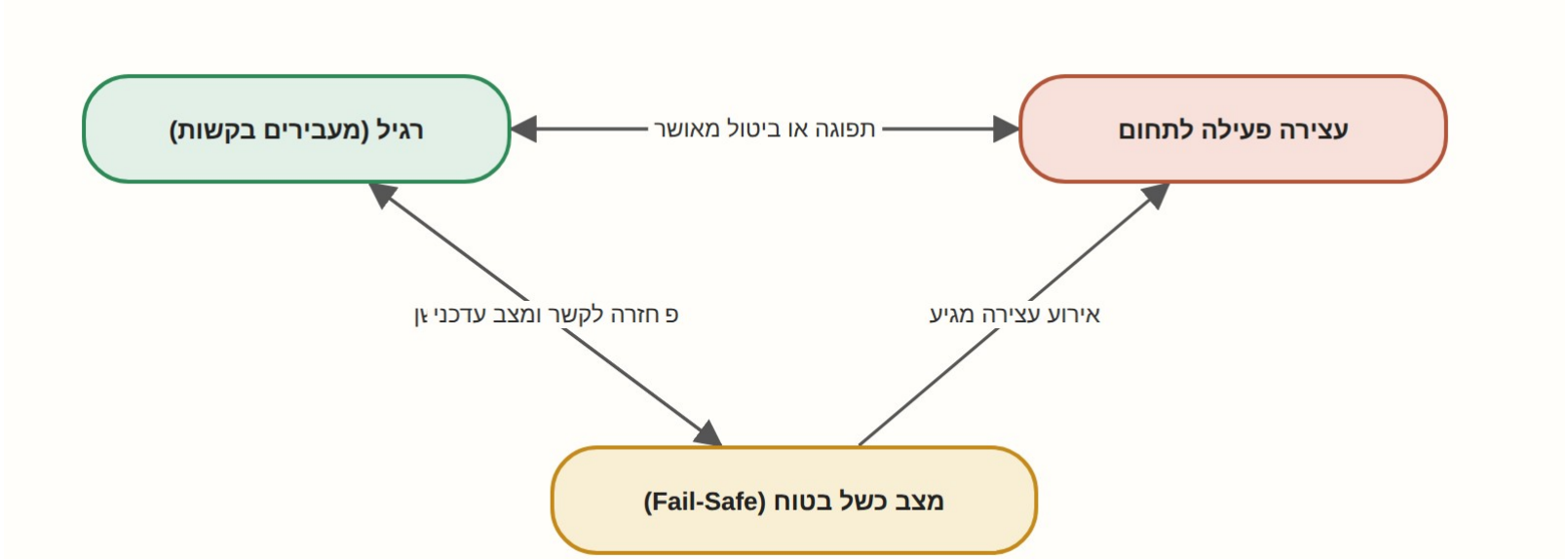
מצב הוא התשובה לשאלה "איפה זה עומד עכשיו". הגדרת המצבים והמעברים מראש מונעת מצבים לא מוגדרים, כמו פקודה שפורסמה בלי שנרשמה.

מחזור החיים של האובייקט המרכזי



מעברי מצב. ירוק הוא סיום תקין, אדום הוא דחייה, צהוב הוא מצב ביניים.

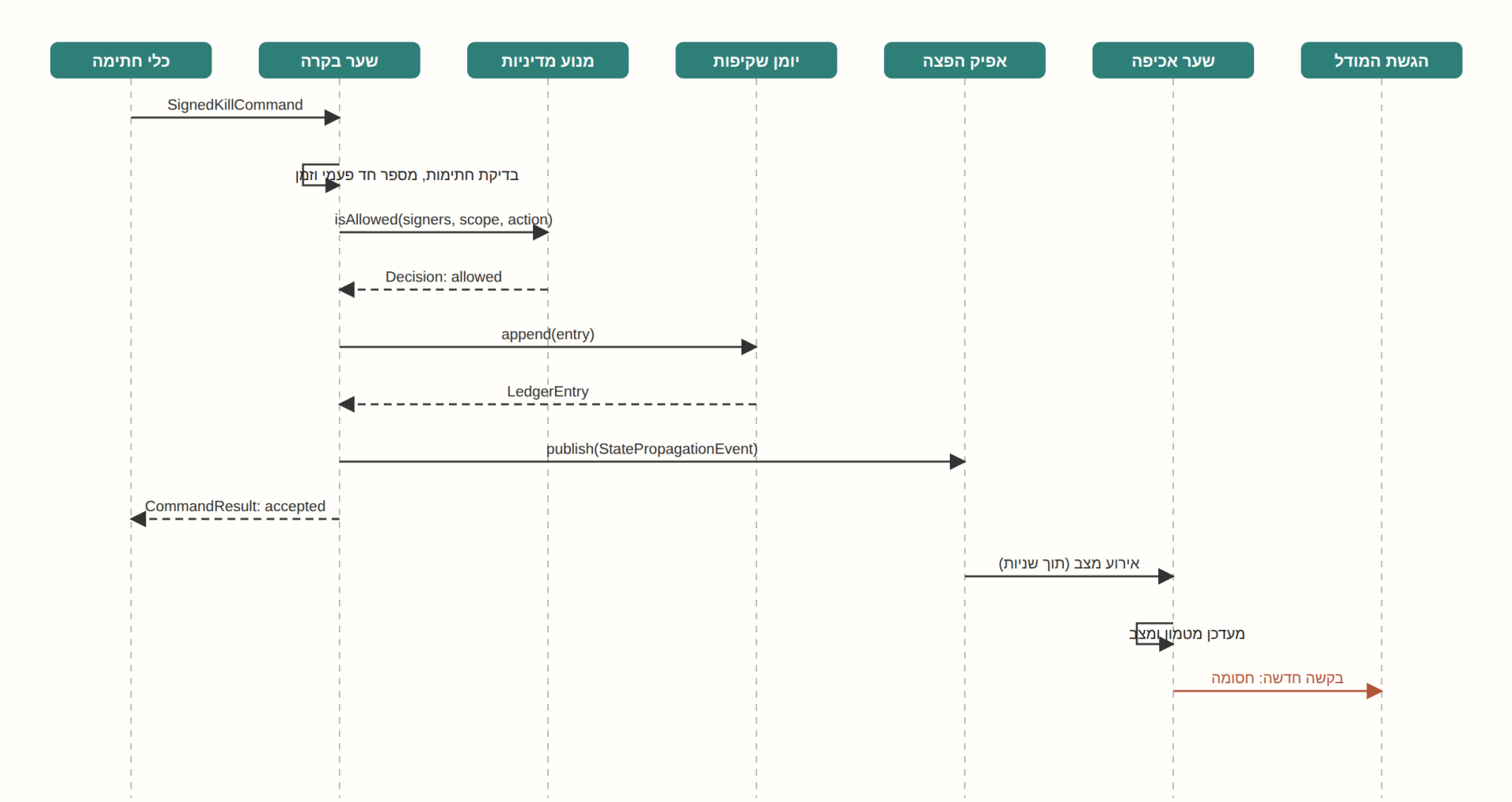
המצב של הרכיב שמגיב לסבירה



מצבים ומעברים

4. דיאגרמת רצף

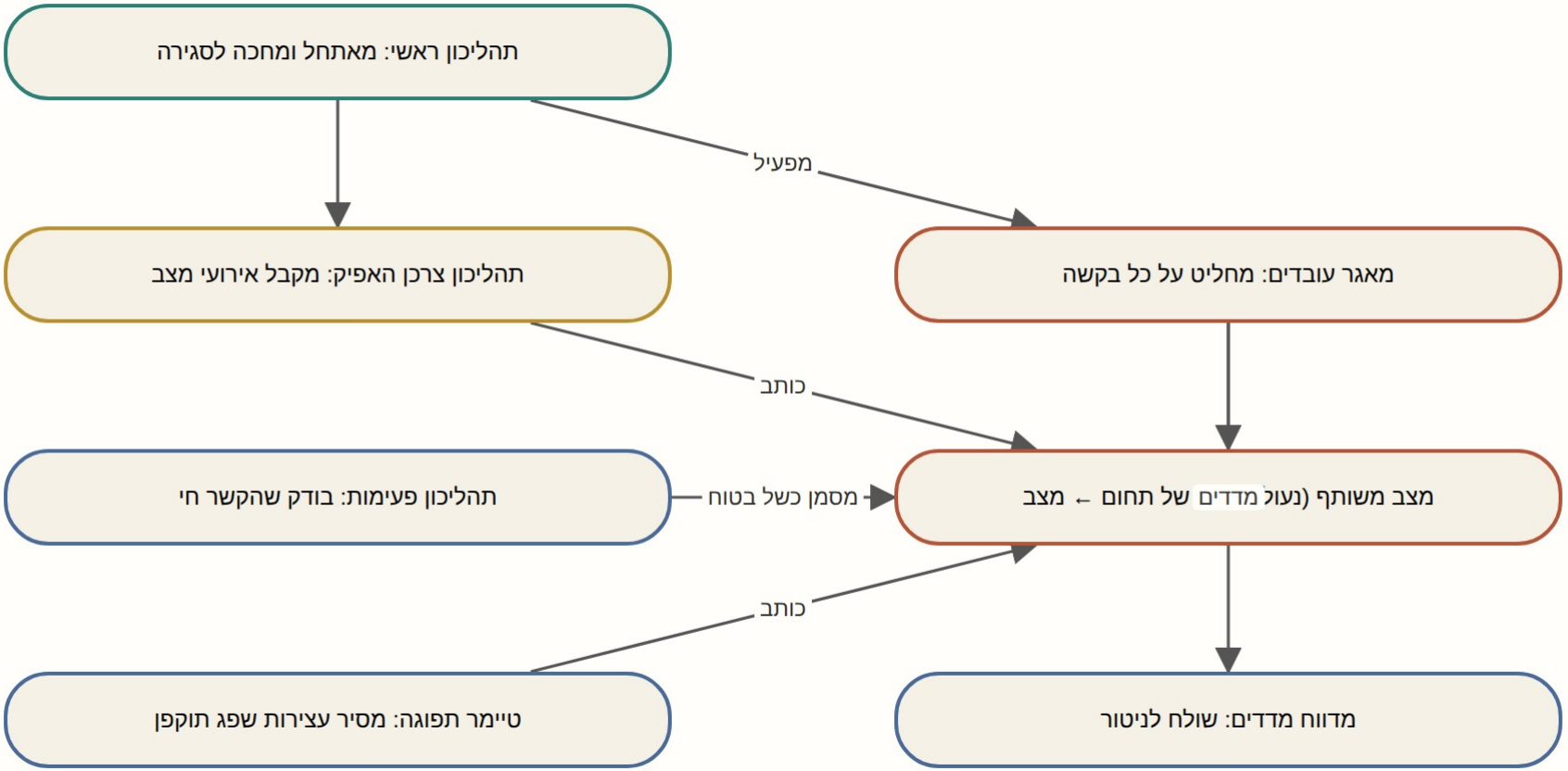
מי קורא למי ובאיזה סדר, מלמעלה למטה. חץ רגיל הוא קריאה, חץ מקווקו הוא תשובה, וחץ אדום הוא תוצאה שחוסמת.



הרצף התקין מקצה לקצה

5. מודל תהליכונים

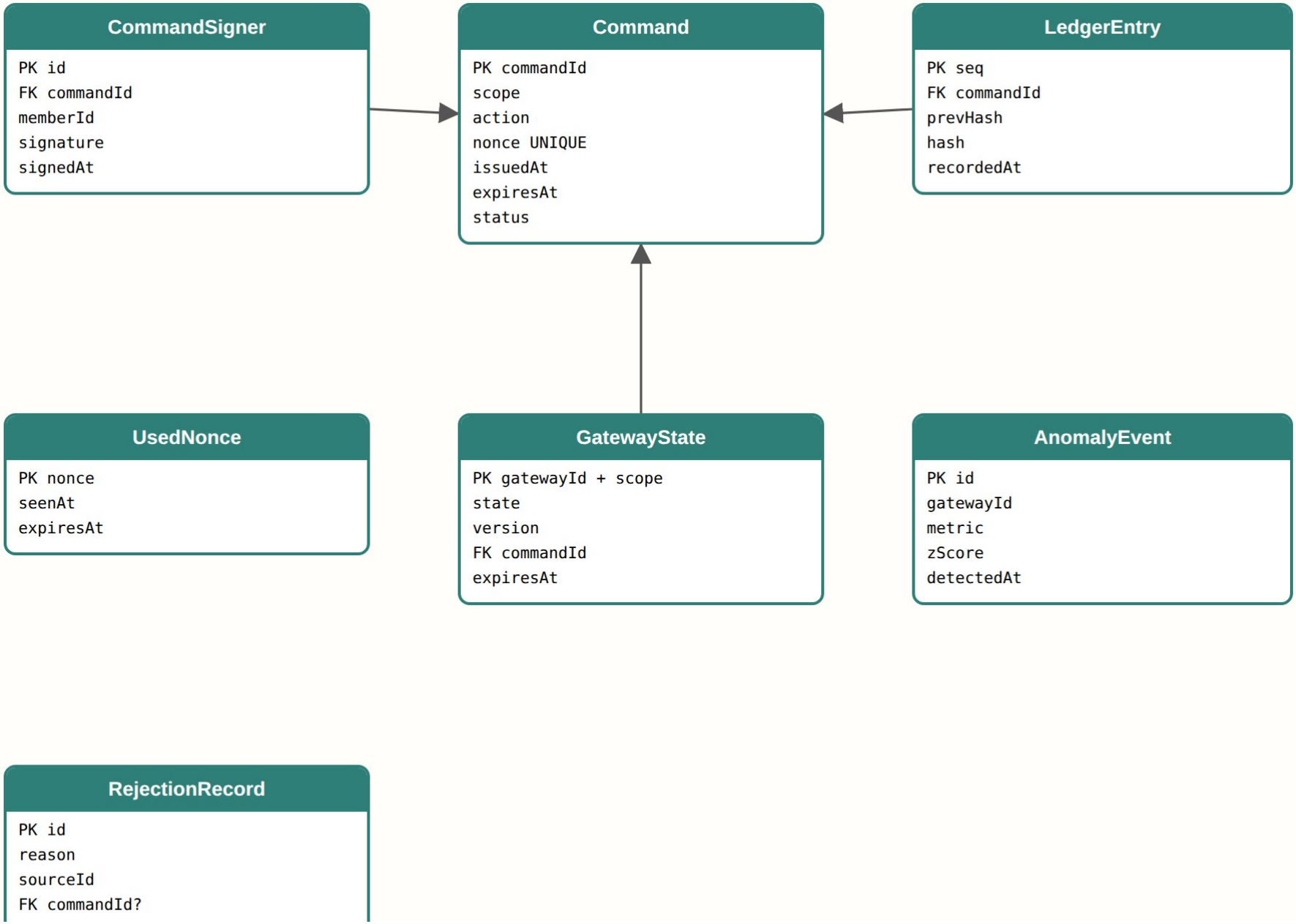
תהליכון הוא "עובד" שרץ במקביל לאחרים. כשכמה תהליכונים נוגעים באותו מצב, חייבים נעילה, אחרת שניים עלולים לשנות אותו באותו רגע ולהשאיר אותו שבור.



תהליכונים ומה שהם חולקים

6. עיצוב טבלאות

מה נשמר לטווח ארוך, ואיך הטבלאות מקושרות. PK הוא מפתח ראשי שמזהה שורה, ו-FK הוא הפניה לשורה בטבלה אחרת.


CommandSigner
PK id
FK commandId
memberId
signature
signedAt
Command
PK commandId
scope
action
nonce UNIQUE
issuedAt
expiresAt
status
LedgerEntry
PK seq
FK commandId
prevHash
hash
recordedAt
UsedNonce
PK nonce
seenAt
expiresAt
GatewayState
PK gatewayId + scope
state
version
FK commandId
expiresAt
AnomalyEvent
PK id
gatewayId
metric
zScore
detectedAt
RejectionRecord
PK id
reason
sourceId
FK commandId?

7. סדר המימוש לפי גרסאות

- **V1.0.0 שלד:** SigningClient, SignedKillCommand, ControlGateway ושער אכיפה פשוט. פקודה אחת חתומה עוצרת מודל אחד.
- **V2.0.0 אימות:** CommandValidator, NonceStore, IPolicyEngine וחתימת סף של שלושה מתוך חמישה.
- **V3.0.0 שקיפות והפצה:** IAppendOnlyLog, IStateBus ואזורים מרובים.
- **V4.0.0 עמידות:** StateCache, HeartbeatMonitor, מצב כשל בטוח ותפוגה אוטומטית.
- **V5.0.0 תצפית:** SelfMonitor, AuditSink ויומן אבטחה מרכזי.